

Лабораторная работа №8

Целочисленная арифметика многократной точности

12 Марта 2026 - Ли Хан

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Основная проблема современных вычислений в криптографии заключается в том, что стандартные типы данных ограничены по размеру. В данной работе я реализовал пять алгоритмов, которые позволяют обрабатывать числа любой длины, представляя их в виде массивов цифр в системе счисления с основанием b .

Детальный разбор алгоритмов

Этот алгоритм выполняет поразрядное суммирование, имитируя расчет «в столбик».

- Логика кода: В программе реализован цикл, который вычисляет сумму `current_sum = u[j] + v[j] + k`. Переменная `k` является ядром переноса: она принимает значение 1, если сумма превышает основание `b`, и 0 в противном случае.
- Реализация функции: Код использует операцию `% b` для записи текущего разряда и `// b` для обновления переноса `k` для следующего шага. Это гарантирует, что избыток суммы корректно передается в старший разряд.

Алгоритм реализует операцию $u - v$, где $u \geq v$, с использованием логики «заема».

- Логика кода: Переменная k здесь работает как счетчик заема. Программа вычисляет `current_diff = u[j] - v[j] + k`.
- Реализация функции: Если результат вычитания отрицателен, операция `% b` в коде автоматически заимствует единицу из старшего разряда, а `math.floor(current_diff / b)` устанавливает $k = -1$ для следующей итерации. Это позволяет коду корректно обрабатывать последовательные займы.

Это классическая реализация длинного умножения через вложенные циклы.

- Логика кода: Внешний цикл фиксирует цифру множителя v_j , а внутренний перемножает её со всеми цифрами u_i . Ключевая строка $t = u[i] * v[j] + w[i+j+1] + k$ выполняет суммирование со смещением.
- Реализация функции: Индекс $i+j+1$ в коде автоматически позиционирует частичное произведение в правильный разряд итогового массива, объединяя умножение, накопление предыдущих сумм и обработку переноса k в одной операции.

Алгоритм 4: Быстрый столбик

Оптимизированный метод, минимизирующий обращения к массиву за счет использования накопителя.

- Логика кода: Вместо пошагового обновления ячеек, код использует одну переменную-аккумулятор t . Для каждого целевого разряда результата s программа собирает в t вклады всех пар (u_i, v_j) , сумма индексов которых соответствует s .
- Реализация функции: Перенос обрабатывается только один раз для всего разряда целиком через $t \gg b$. Это значительно ускоряет выполнение, так как код записывает окончательное значение в ячейку массива только после завершения всех суммирований для данного разряда.

Наиболее сложный модуль, использующий метод пробных цифр частного.

- Логика кода: Процесс основан на цикле «оценка-исправление». Код сначала оценивает цифру частного q , деля старшие разряды делимого на старший разряд делителя.
- Реализация функции: В коде реализован блок коррекции: если произведение $q \cdot v$ оказывается больше текущего остатка, программа уменьшает q и восстанавливает остаток путем прибавления делителя. Это гарантирует получение максимально точной цифры частного на каждом шаге.

Итоги работы

Программная реализация этих пяти алгоритмов позволяет нам полностью абстрагироваться от ограничений процессора. Весь контроль над математической логикой (переносы, займы, оценки) перенесен в код, что дает возможность проводить вычисления с числами, состоящими из тысяч знаков, что критически важно для криптографии.